

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

REPORT

Laboratory Work No. 8

Operating Systems

The Same Calls, Inside Your Own OS (xv6)

Student: Bianca-Andreea Tataroi
Group: FAF-242
Checked by: Patricia Reițman

Chişinău – 2026

Contents

1 Objective **2**

2 Laboratory Environment **2**

3 Part 1 – fork, exec, and wait in xv6 **2**

3.1 Creating and executing run.c with echo 2

3.2 Replacing echo with ls 4

3.3 Testing an execution failure 5

3.4 Part 1 – Observations 5

4 Part 2 – Two-way communication with pingpong **6**

4.1 Creating pingpong.c 6

4.2 Executing pingpong 6

4.3 Part 2 – Observations 7

5 Results and Discussion **7**

6 GitHub Repository and Submission Files **8**

7 Conclusion **9**

1 Objective

The objective of Laboratory Work No. 8 was to reproduce process-management operations from Linux inside xv6, a small teaching operating system, and to explore interprocess communication using pipes. In particular, this laboratory demonstrates the sequence `fork()`, `exec()`, and `wait()`, and implements a two-way message exchange between a parent and child process.

The mandatory activities were: (1) running the `run.c` program with `echo`, `ls`, and a nonexistent executable; (2) implementing `pingpong.c` using two pipes; and (3) answering the observation questions in `notes.md`. The optional `primes.c` activity was not included.

2 Laboratory Environment

The work was performed in an Ubuntu virtual machine using the existing xv6-RISC-V source tree at `/home/bianca/xv6-riscv`. The Nano editor was used to create the programs, the RISC-V cross-compiler built the xv6 executables, and QEMU was used to boot the operating system.

```
cd ~/xv6-riscv
nano user/run.c
nano Makefile
make qemu
```

For each new program, its C source was placed in the `user/` directory and an entry was added to the `UPROGS` list in the Makefile:

```
$U/_run\
$U/_pingpong\
```

The xv6 shell uses the `$` prompt. Programs were launched by typing `run` or `pingpong` there, without `./`. The corresponding build completed and xv6 booted successfully.

3 Part 1 – fork, exec, and wait in xv6

3.1 Creating and executing run.c with echo

The first program reproduces the three key steps of starting another program. The parent prints its PID, `fork()` creates a child, `exec()` changes the child's program to `echo`, and `wait()` collects the child's exit status.

The relevant code in `user/run.c` was:

```
printf("parent: my pid is %d\n", getpid());
int pid = fork();
if(pid < 0){
    printf("fork failed\n");
    exit(1);
}
if(pid == 0){
    printf("child: my pid is %d\n", getpid());
    char *args[] = {"echo", "hello", "from", "the", "child", 0};
    exec("echo", args);
    printf("exec failed\n");
    exit(1);
}
int status;
wait(&status);
printf("parent: child %d ended with status %d\n", pid, status);
exit(0);
```

The program was executed within xv6:

```
$ run
parent: my pid is 3
child: my pid is 4
hello from the child
parent: child 4 ended with status 0
```

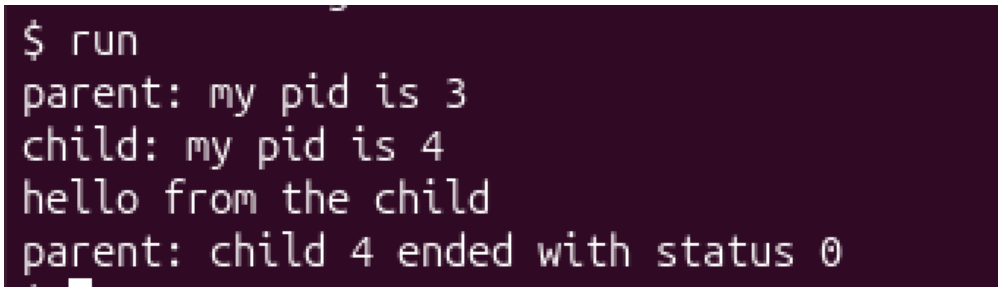


Figure 1: Successful execution of the echo program in the child process.

The message was printed by `echo` after `exec()` succeeded. The value 0 indicates that the child terminated successfully.

3.2 Replacing echo with ls

The child's argument array and executable name were changed to run `ls` instead of `echo`:

```
char *args[] = {"ls", 0};
exec("ls", args);
```

After leaving QEMU, recompiling with `make qemu`, and running `run` again, the xv6 directory contents were printed. The listing included system programs such as `cat`, `echo`, `ls`, and the new `run` executable. The parent then reported status 0.

```
$ run
parent: my pid is 3
child: my pid is 4
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 36728
echo           2 4 35592
forktest       2 5 18136
grep           2 6 44184
init           2 7 35976
kill           2 8 35512
ln             2 9 35304
ls             2 10 42880
mkdir          2 11 35568
rm             2 12 35552
sh             2 13 58312
stressfs       2 14 36416
usertests      2 15 206800
grind          2 16 51824
wc             2 17 37728
zombie         2 18 34848
logstress      2 19 37560
forphan        2 20 36352
dorphan        2 21 35800
sync           2 22 34944
sleep          2 23 35280
args           2 24 35272
mystat         2 25 36320
find           2 26 38080
run            2 27 35888
console        3 28 0
parent: child 4 ended with status 0
```

Figure 2: The child executes ls and displays the contents of the xv6 filesystem.

This shows that the parent/child control flow stays the same even when the child executes a different program.

3.3 Testing an execution failure

Next, the child was instructed to execute a nonexistent program:

```
char *args[] = {"nosuchprog", 0};
exec("nosuchprog", args);
printf("exec failed\n");
exit(1);
```

The observed output was:

```
$ run
parent: my pid is 3
child: my pid is 4
exec failed
parent: child 4 ended with status 1
```

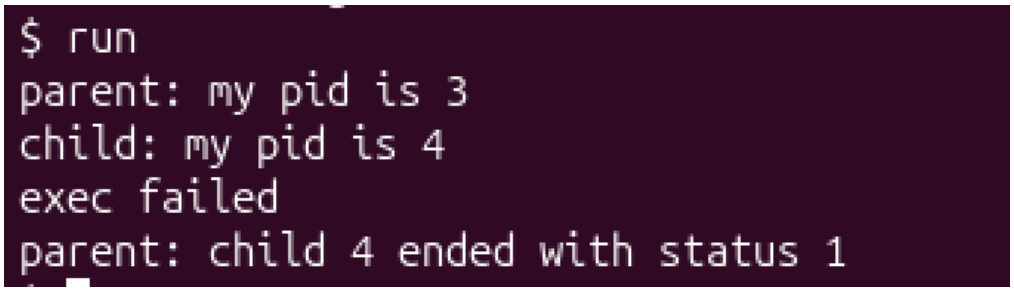


Figure 3: Failed exec call for a nonexistent program, ending with status 1.

When `exec()` fails, it returns to the calling program, so the next statement prints `exec failed`. The explicit `exit(1)` produces the status subsequently collected by the parent. After this experiment, the source was restored to the original working `echo` version.

3.4 Part 1 – Observations

1. **Which three calls are used in xv6 and in Linux Lab 7? What is the same?** Both exercises use `fork()`, an `exec`-family call, and a wait operation. In Lab 7, the Linux program used `execlp()` and `waitpid()`, while in Lab 8 the xv6 program uses `exec()` and `wait()`. Their underlying roles are the same: create a child process, replace its program, and collect its termination status.

2. **Who prints "hello from the child"?** The `echo` program prints the message. The original `run.c` child invokes `exec()` to become `echo`.

3. **Why does the failed execution return status 1 in xv6 but 127 in Linux**

Lab 7? The difference comes from the explicit failure paths: Lab 8 calls `exit(1)`, whereas Lab 7 called `exit(127)`. Neither value is automatically assigned just because `exec` failed.

4 Part 2 – Two-way communication with pingpong

4.1 Creating pingpong.c

The second mandatory program used two pipes, one for each communication direction. `p2c` carries a byte from parent to child; `c2p` carries a reply from child to parent. Each pipe has a reading end at index 0 and a writing end at index 1.

The key part of `user/pingpong.c` was:

```
int p2c[2], c2p[2];
char b = 'x';
pipe(p2c);
pipe(c2p);
if(fork() == 0){
    close(p2c[1]); close(c2p[0]);
    read(p2c[0], &b, 1);
    printf("%d: received ping\n", getpid());
    write(c2p[1], &b, 1);
    close(p2c[0]); close(c2p[1]);
    exit(0);
}
close(p2c[0]); close(c2p[1]);
write(p2c[1], &b, 1);
read(c2p[0], &b, 1);
printf("%d: received pong\n", getpid());
close(p2c[1]); close(c2p[0]);
wait(0);
exit(0);
```

The unused pipe endpoints are closed in each process, and the remaining endpoints are closed after the exchange.

4.2 Executing pingpong

After adding `$U/_pingpong` to the Makefile and rebuilding xv6, the program was run:

```
$ pingpong
4: received ping
3: received pong
```

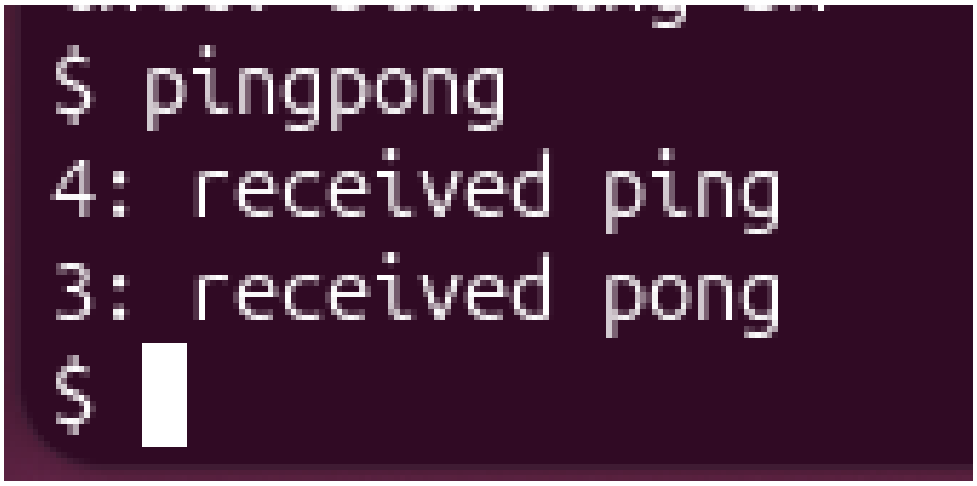


Figure 4: The child receives ping before the parent receives pong.

The child prints first because the parent blocks on `read(c2p[0], &b, 1)` until the child writes the reply. The process IDs can change between runs.

4.3 Part 2 – Observations

1. **Why are two pipes needed instead of one?** Each pipe is used as a one-way channel. One carries the request from the parent to the child, while the other carries the response from the child to the parent. This makes the two communication directions explicit and avoids having both processes compete for data in one pipe.

2. **Which call makes the parent wait for the reply?** The blocking `read(c2p[0], &b, 1)` call waits until the child writes a byte into the response pipe. The later `wait(0)` call serves a different purpose: it waits for the child to terminate.

5 Results and Discussion

Both mandatory programs built successfully and produced the expected results. The `run` experiment showed successful child execution with exit status 0, and a failed `exec` with explicit exit status 1. The `pingpong` experiment showed communication in both directions, with the child receiving `ping` before the parent received `pong`.

The two programs were also run one after another in the final verified xv6 session:

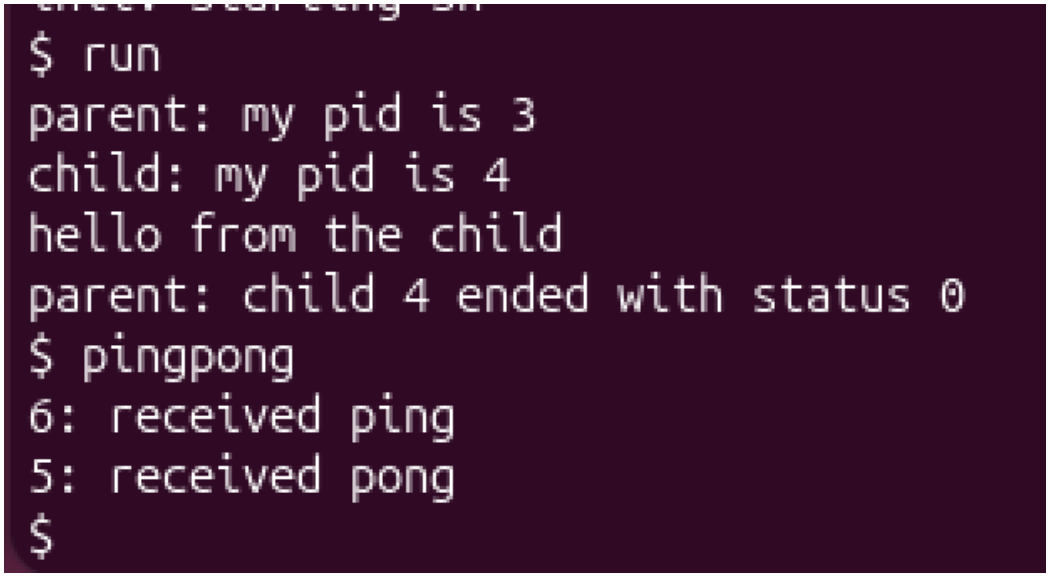


Figure 5: Final verification: run and pingpong both execute successfully in xv6.

6 GitHub Repository and Submission Files

The four required deliverable files were copied from the xv6 tree into the separate `os-lab8` folder:

```
os-lab8/
  Makefile
  notes.md
  user/
    run.c
    pingpong.c
```

The local repository was initialized and committed as follows:

```
git init
git branch -M main
git add .
git commit -m "Complete Operating Systems Lab 8"
```

The local commit identifier was `d1f1ccd`, and the working tree was clean. The configured remote URL was:

https://github.com/Biancatataroi/Operating-Systems-Lab8

At the time of preparing this report, the remote push had *not yet been confirmed*: an attempted push returned HTTP 403 (permission denied). Therefore, a successful online

submission cannot yet be claimed. Additionally, the `os-lab8` folder holds only the changed files, not a standalone buildable xv6 tree; testing was carried out in `/xv6-riscv`.

7 Conclusion

In this laboratory work, I implemented and tested process management and interprocess communication directly inside xv6. The first exercise illustrated how a parent creates a child with `fork()`, how the child changes its running program with `exec()`, and how the parent collects the result with `wait()`. Changing the child program from `echo` to `ls` demonstrated that the same process-control structure can launch different executables. Intentionally requesting a nonexistent program made the `exec()` failure path and the meaning of `exit(1)` clear.

The second exercise showed two-way communication using two pipes. By placing blocking `read()` and `write()` calls in the appropriate processes and closing unused endpoints, the parent and child completed a predictable ping-pong exchange. Comparing this laboratory with Lab 7 also highlighted that xv6 and Linux follow similar process-management concepts even though they provide different interfaces and runtime environments.